

Learn C The Hard Way – 2

남궁명수

Exercise 21. 고급 데이터 타입과 흐름 제어

이번 장은 C에서 사용 가능한 모든 데이터 타입과 흐름 제어 구조를 정리한 완전한 참고자료가 될 것이다.

이것은 너의 지식을 완성하기 위한 참고서 역할을 하며, 입력해야 할 코드는 없다.

나는 네가 중요한 개념을 확실히 머릿속에 정착시키도록 플래시 카드를 만들어 일부 내용을 암기하게 할 것이다.

이 연습이 유용하려면, 최소 일주일엔 이 내용을 반복해서 익히고, 여기에서 빠져 있는 모든 요소를 소스로 채워 넣어야 한다.

각 항목이 무엇을 의미하는지 직접 써보고, 조사한 내용을 확인하기 위해 프로그램도 작성하게 될 것이다.

[21-1] 사용 가능한 데이터 타입

[1] 기본 정수 타입

타입	설명
char	문자형 (1바이트, 구현에 따라 signed/unsigned)
signed char	부호 있는 char
unsigned char	부호 없는 char
short	짧은 정수 (최소 16비트)
short int	short와 동일
signed short	부호 있는 short
unsigned short	부호 없는 short
int	기본 정수형
signed int	부호 있는 int
unsigned int	부호 없는 int
long	긴 정수
long int	long과 동일
signed long	부호 있는 long
unsigned long	부호 없는 long
long long	더 긴 정수 (C99)
unsigned long long	부호 없는 long long

[2] 실수 타입

타입	설명
float	단정밀도 부동소수점
double	배정밀도 부동소수점
long double	확장 정밀도 부동소수점

[3] 기타 기본 타입

타입	설명
void	타입 없음
_Bool	불리언 (C99)
size_t	객체 크기 표현용
ptrdiff_t	포인터 차이 표현용

[21-2] 타입 수식어

수식어	설명
signed	부호 있음
unsigned	부호 없음
short	더 작은 정수
long	더 큰 정수

예:

unsigned long int

[21-3] 타입 한정자

한정자	설명
const	값 변경 불가
volatile	최적화 금지
restrict	포인터 최적화 관련 (C99)
_Atomic	원자적 접근 (C11)

[21-4] 타입 변환

C는 일종의 단계적 타입 승격 메커니즘을 사용한다.

표현식의 양쪽 피연산자를 보고, 연산을 수행하기 전에 더 작은 쪽을 더 큰 쪽 타입에 맞춰서 승격시킨다.

표현식의 한쪽이 아래 목록 중 하나라면, 다른 쪽은 연산 전에 해당 타입으로 변환된다. 순서는 다음과 같다.

1. long double
2. double
3. float
4. int(단, char 와 short int 만 해당)
5. long

만약 표현식에서 타입 변환이 어떻게 이루어지는지 고민하게 된다면,
컴파일러에게 맡기지 말라.
명시적 캐스팅을 사용해서 정확히 원하는 타입으로 만들어라.

예를 들어 다음과 같은 식이 있다면:

```
long + char - int * double
```

이것이 올바르게 double 로 변환될지 고민하지 말고, 다음처럼 캐스팅하라.

```
(double)long - (double)char - (double)int * double
```

원하는 타입을 괄호 안에 넣어 변수 앞에 붙이면 강제로 그 타입으로 변환할 수 있다.
하지만 중요한 것은 항상 위로 승격해야 한다는 점이다.
내려서(downcast) 변환하지 말라.
long 을 char 로 캐스팅하지 말라. 정확히 무엇을 하는지 모른다면.

[21-5] stdint.h 정확한 크기 타입

stdint.h 는 정확한 크기의 정수 타입들을 위한 typedef 집합과, 모든 타입의 크기를 나타내는 매크로 집합을 정의한다.

이는 예전의 limits.h 보다 일관성이 있어 사용하기 쉽다. 여기 정의된 타입들의 패턴은 다음과 같다.

형식:

```
(u)int(BITS)_t
```

앞에 u 가 붙으면 “unsigned”를 의미한다.
BITS 는 비트 수를 나타낸다.

예:

```
uint32_t, int16_t
```

이 패턴은 해당 타입의 최대값을 반환하는 매크로에도 반복된다.

[1] 정확한 크기 정수

타입	설명
int8_t	8비트 signed
uint8_t	8비트 unsigned
int16_t	16비트 signed
uint16_t	16비트 unsigned
int32_t	32비트 signed
uint32_t	32비트 unsigned
int64_t	64비트 signed
uint64_t	64비트 unsigned

[2] 최소 크기 정수

타입	설명
int_least8_t	최소 8비트
uint_least8_t	최소 8비트 unsigned
int_least16_t	최소 16비트
int_least32_t	최소 32비트
int_least64_t	최소 64비트

[3] 가장 빠른 정수 타입

타입	설명
int_fast8_t	최소 8비트, 가장 빠른 타입
int_fast16_t	최소 16비트
int_fast32_t	최소 32비트
int_fast64_t	최소 64비트

[4] 포인터 관련 타입

타입	설명
intptr_t	포인터 저장 가능한 signed
uintptr_t	포인터 저장 가능한 unsigned

[21-6] 타입 한계 매크로(stdint.h)

매크로	설명
INT8_MAX	int8_t 최대값
INT8_MIN	int8_t 최소값
UINT8_MAX	uint8_t 최대값
INT16_MAX	16비트 최대
INT32_MAX	32비트 최대
INT64_MAX	64비트 최대
SIZE_MAX	size_t 최대값

패턴:

INT(N)_MAX

- (N)비트 signed 정수의 최대 양수 값
- INT16_MAX

INT(N)_MIN

- (N)비트 signed 정수의 최소 음수 값

UINT(N)_MAX

- (N)비트 unsigned 정수의 최대값
- unsigned 이므로 최소값은 0 이고 음수는 없다.

주의하자

- 헤더 파일에서 실제로 INT(N)_MAX 라는 리터럴 정의를 찾으려고 하지 말라. 여기서 (N)은 현재 플랫폼이 지원하는 비트 수를 나타내는 자리표시자이다.
- 이(N)은 8, 16, 32, 64 심지어 128 일 수도 있다. 이 장에서는 모든 조합을 일일이 쓰지 않기 위해 이런 표기법을 사용한다.

또한 stdint.h 에는 다음과 같은 매크로들도 있다.

- size_t 타입의 크기 관련 매크로
- 포인터를 저장할 수 있을 만큼 충분히 큰 정수 타입
- 기타 유용한 크기 정의 매크로

컴파일러는 최소한 이들 타입을 제공해야 하며, 더 큰 타입을 추가로 지원할 수도 있다.

[21-7] 전체 연산자 표

[1] 수학 연산자

연산자	설명
+	덧셈
-	뺄셈
*	곱셈
/	나눗셈
%	나머지
++	증가
--	감소
()	함수 호출

[2] 데이터 접근 연산자

연산자	설명
[]	배열 인덱스
.	구조체 멤버
->	포인터 구조체 멤버
&	주소 연산
*	역참조
sizeof	크기 반환

[3] 논리/비교 연산자

연산자	설명
==	같다
!=	다르다
>	크다
<	작다
>=	이상
<=	이하

[4] 비트 연산자

연산자	설명
&	AND
	OR
^	XOR
~	NOT
<<	왼쪽 시프트
>>	오른쪽 시프트

[5] 불리언 연산자

연산자	설명
&&	논리 AND
	논리 OR
!	논리 NOT
?:	삼항 연산자

[6] 대입 연산자

연산자	설명
=	대입
+=	더하고 대입
-=	빼고 대입
*=	곱하고 대입
/=	나누고 대입
%=	나머지 대입
&=	비트 AND 대입
=	비트 OR 대입
^=	XOR 대입
<<=	왼쪽 시프트 대입
>>=	오른쪽 시프트 대입

[21-8] 전체 제어 구조

구조	설명
if	조건문
else	조건 분기
switch	다중 선택
case	switch 내부
default	기본 케이스
for	반복문
while	반복문
do-while	최소 1회 실행
break	루프 탈출
continue	다음 반복
goto	레이블 점프
return	함수 종료

[21-9] 추가 과제(Extra Credit)

- stdint.h 를 읽거나 설명을 찾아보고, 사용 가능한 모든 크기 식별자를 적어라.
- 여기 있는 각 항목이 코드에서 무엇을 하는지 직접 작성하자. 온라인에서 조사하여 정확한지 확인하자.
- 플래시 카드를 만들어 하루 15 분씩 연습하며 암기하자.
- 각 타입의 예제를 출력하는 프로그램을 작성하고, 조사한 내용이 맞는지 확인하라.

Exercise 22. 스택, 스코프, 그리고 전역 변수

스코프라는 개념은 프로그래밍을 처음 시작할 때 많은 사람들을 혼란스럽게 만든다.

이 개념은 원래 시스템 스택(stack)의 사용에서 비롯되었고, 스택이 임시 변수를 저장하는 데 사용되던 방식과 관련이 있다.

이번 장에서는 스택 자료구조가 어떻게 동작하는지 배우고, 그 개념을 현대 C에서의 스코프 개념에 다시 연결해보겠다.

하지만 이 장의 진짜 목적은 도대체 C에서 것들이 어디에 존재하는지 배우는 것이다. 누군가 스코프를 이해하지 못하는 경우, 거의 항상 그 원인은 변수가 어디에서 생성되고, 어디에 존재하며, 언제 사라지는지에 대한 이해 부족이다.

일단 “것들이 어디에 있는지”알게 되면, 스코프 개념은 훨씬 쉬워진다.

이 연습에는 세 개의 파일이 필요하다.

- ex22.h : 외부 변수와 함수들을 설정하는 헤더 파일
- ex22.c : 일반적인 main 파일이 아니라, ex22.h 에 정의된 함수와 변수를 포함하는 오브젝트 파일 ex22.o 가 될 소스 파일
- ex22_main.c : 실제 main 함수가 있는 파일로, 위의 두 파일을 포함하고, 그 안의 내용과 다른 스코프 개념을 시연한다.

[22-1] ex22.h

```
#ifndef _ex22_h
#define _ex22_h

// ex22.c 에 정의된 THE_SIZE 변수를
// 다른 .c 파일에서도 사용할 수 있도록 외부 선언
extern int THE_SIZE;

// ex22.c 내부의 static 변수 THE_AGE 를
// 간접적으로 접근하기 위한 함수들
int get_age();
void set_age(int age);

// 함수 내부 static 변수를 사용하는 함수
// 이전 ratio 값을 반환하고 새로운 값으로 갱신
double update_ratio(double ratio);

// 현재 THE_SIZE 값을 출력하는 함수
void print_size();
```

```
#endif
```

[22-2] ex22.c

```
#include <stdio.h>
#include "ex22.h"
#include "dbg.h"

// 전역 변수 (다른 파일에서 extern 으로 접근 가능)
int THE_SIZE = 1000;

// 파일 내부에서만 사용 가능한 static 전역 변수
// 다른 .c 파일에서는 직접 접근 불가능
static int THE_AGE = 37;

// THE_AGE 값을 반환하는 함수
int get_age()
{
    return THE_AGE;
}

// THE_AGE 값을 변경하는 함수
void set_age(int age)
{
    THE_AGE = age;
}

// 함수 내부 static 변수 사용 예제
// 이전 ratio 값을 반환하고 새 값으로 변경
double update_ratio(double new_ratio)
{
    // 함수가 종료되어도 값이 유지되는 static 변수
    static double ratio = 1.0;

    // 기존 값을 저장
    double old_ratio = ratio;

    // 새로운 값으로 갱신
    ratio = new_ratio;
```

```

    // 이전 값 반환
    return old_ratio;
}

// 현재 THE_SIZE 값을 출력
void print_size()
{
    log_info("I think size is: %d", THE_SIZE);
}

```

[22-3] ex22_main.c

```

#include "ex22.h"
#include "dbg.h"

// 상수 문자열 (읽기 전용)
const char *MY_NAME = "Zed A. Shaw";

// 스코프(범위) 동작을 확인하기 위한 함수
void scope_demo(int count)
{
    // 함수 시작 시 전달받은 count 값 출력
    log_info("count is: %d", count);

    // 새로운 블록 시작 (if 문 내부는 별도의 스코프)
    if (count > 10) {

        // ⚠️ 기존 count 와는 완전히 다른 새로운 변수
        // 블록 내부에서만 존재
        int count = 100;

        log_info("count in this scope is %d", count);
    }

    // 위 블록이 끝났으므로
    // 여기서의 count 는 함수 파라미터 count
    log_info("count is at exit: %d", count);

    // 함수 내부에서 파라미터 값 변경
    count = 3000;
}

```

```

    log_info("count after assign: %d", count);
}

int main(int argc, char *argv[])
{
    // THE_AGE 접근 테스트 (간접 접근)
    log_info("My name: %s, age: %d", MY_NAME, get_age());

    // 나이 변경
    set_age(100);

    log_info("My age is now: %d", get_age());

    // extern 변수 THE_SIZE 테스트
    log_info("THE_SIZE is: %d", THE_SIZE);

    // ex22.c 내부에서 바라보는 값 출력
    print_size();

    // 전역 변수 값 변경
    THE_SIZE = 9;

    log_info("THE SIZE is now: %d", THE_SIZE);

    // 변경된 값 확인
    print_size();

    // 함수 내부 static 변수 동작 확인
    log_info("Ratio at first: %f", update_ratio(2.0));
    log_info("Ratio again: %f", update_ratio(10.0));
    log_info("Ratio once more: %f", update_ratio(300.0));

    // 스코프 테스트
    int count = 4;

    scope_demo(count);          // 4 전달
    scope_demo(count * 20);     // 80 전달

    // 함수 내부에서 변경해도
    // 여기의 count 는 영향받지 않음
    log_info("count after calling scope_demo: %d", count);
}

```

```
}  
    return 0;  
}
```

[22-4] 실행 결과 예시

Makefile 대신 직접 빌드하기

```
cc -Wall -g -DNDEBUG -c -o ex22.o ex22.c  
cc -Wall -g -DNDEBUG ex22_main.c ex22.o -o ex22_main  
./ex22_main
```

[22-5] Scope, Stack, and Bugs

이제 다양한 위치에 변수를 배치할 수 있다는 것을 알았을 것이다.

- extern 또는 접근 함수로 전역 생성
- 블록 내부 변수 생성
- 함수 매개변수 변경
- static 변수 유지

하지만 이 모든 것은 버그를 만든다.

C는 변수를 여러 위치에 배치하고 접근하게 하므로, 어디에 있는지 모르면 관리 실패 가능성이 높다.

Exercise 22-1. 모듈 구조 및 동작 설명서

[22-1] 개요

본 예제는 C 언어에서의 다음 개념들을 종합적으로 설명하기 위한 모듈 구조이다.

- 헤더 파일과 소스 파일의 역할 분리
- extern 을 통한 전역 변수 공유
- static 을 이용한 정보 은닉
- 함수 내부 static 변수의 동작 원리
- 스코프(scope)와 변수 새도잉(shadowing)
- 값 전달(call by value)

본 모듈은 다음 세 파일로 구성된다.

ex22.h	→ 인터페이스 선언
ex22.c	→ 실제 구현
ex22_main.c	→ 사용 예제 (main 함수 포함)

[22-2] ex22.h(인터페이스 정의)

[1] 전체 코드

```
#ifndef _ex22_h
#define _ex22_h

extern int THE_SIZE ;

int get_age();
void set_age(int age);

double update_ratio(double ratio);

void print_size();

#endif _ex22_h
```

[1-1] 헤더 가드

```
#ifndef _ex22_h
#define _ex22_h
...
#endif _ex22_h
```

목적

- 동일 헤더 파일이 여러 번 include 되는 것을 방지
- 중복 선언으로 인한 컴파일 오류 예방

동작 원리

- `_ex22.h` 가 정의되지 않은 경우에만 파일 내용을 포함
- 최초 포함 시 `_ex22.h` 를 정의
- 이후 재포함 시 무시됨

[1-2] 외부 전역 변수 선언

```
extern int THE_SIZE;
```

의미

- `THE_SIZE` 라는 정수형 전역 변수가 다른 소스 파일에 정의되어 있음을 선언
- 여기서는 메모리를 생성하지 않음

특징

선언 형태	메모리 생성 여부
<code>int THE_SIZE;</code>	생성됨
<code>extern int THE_SIZE;</code>	생성되지 않음

[1-3] 함수 선언부

```
int get_age();  
void set_age(int age);  
double update_ratio(double ratio);  
void print_size();
```

역할

- `ex22.c` 내부 기능을 외부에 공개
- 구현 세부 사항은 감추고 인터페이스만 제공

[22-3] ex22.c(구현부)

[1] 전체 코드

```
#include <stdio.h>
#include "ex22.h"
#include "dbg.h"

// 전역 변수(다른 파일에서 extern 으로 접근 가능)
int THE_SIZE = 100;

// 파일 내부에서만 사용 가능한 static 전역 변수
// 다른 .c 파일에서는 직접 접근 불가능
static int THE_AGE = 37;

// THE_AGE 값을 변환하는 함수
int get_age()
{
    return THE_AGE;
}

// THE_AGE 값을 변경하는 함수
void set_age(int age)
{
    THE_AGE = age;
}

// 함수 내부 static 변수 사용 예제
// 이전 ratio 값을 반환하고 새 값으로 변경
double update_ratio(double new_ratio)
{
    // 함수가 종료되어도 값이 유지되는 static 변수
    static double ratio = 1.0;

    // 기존 값을 저장
    double old_ratio = ratio;

    // 새로운 값으로 갱신
    ratio = new_ratio;

    // 이전 값 반환
    return old_ratio;
}
```



```
// 현재 THE_SIZE 값을 출력
void print_size()
{
    log_info("I think size is: %d", THE_SIZE);
}
```

[1-1] 전역 변수 정의

```
int THE_SIZE = 1000;
```

특징

- 실제 메모리 공간 생성
- extern 을 통해 다른 파일에서 접근 가능

[1-2] static 전역 변수

```
static int THE_AGE = 37;
```

특징

- 파일 내부에서만 접근 가능
- 다른 소스 파일에서는 직접 접근 불가
- extern 선언 불가능

목적

- 내부 구현 데이터 보호
- 모듈 단위 캡슐화 구현

[1-3] THE_AGE 접근 함수

get_age()

```
int get_age()
{
    return THE_AGE;
}
```

set_age()

```
void set_age(int age)
{
    THE_AGE = age;
}
```

설계 의도

- static 전역 변수에 대한 간접 접근 제공
- 데이터 보호 및 제어된 수정 가능
- C 언어에서의 정보 은닉 구현

[1-4] 함수 내부 static 변수

```
double update_ratio(double new_ratio)
{
    static double ratio = 1.0;
    double old_ratio = ratio;
    ratio = new_ratio;
    return old_ratio;
}
```

특징

- static 지역 변수는 함수 종료 후에도 값 유지
- 최초 1 회만 초기화
- 프로그램 종료 시까지 메모리에 존재

동작 과정

호출	반환값	내부 ratio 값
update_ratio(2.0)	1.0	2.0
update_ratio(10.0)	2.0	10.0
update_ratio(300.0)	10.0	300.0

[1-5] print_size()

```
void print_size()
{
    log_info("I think size is: %d", THE_SIZE);
}
```

역할

- 현재 전역 변수 THE_SIZE 값 출력
- 동일한 전역 메모리를 참조함을 확인 가능

[22-4] ex22_main.c(사용부)

[1] 전체 코드

```
#include "ex22.h"
#include "dbg.h"

// 상수 문자열(읽기 전용)
const char *MY_NAME = "Zed A. Shaw";

// 스코프(범위) 동작을 확인하기 위한 함수
void scope_demo(int count)
{
    // 함수 시작 시 전달받은 count 값 출력
    log_info("count is: %d\n", count);

    // 새로운 블록 시작
    if(count > 10) {
        // 블록 내부에서만 존재
        int count = 100;
        log_info("count in this scope is %d", count);
    }

    // 위 블록이 끝났으므로
    // 여기서의 count 는 함수 파라미터 count
    log_info("count is at exit: %d", count);

    // 함수 내부에서 파라미터 값 변경
    count = 300;
    log_info("count after assing: %d", count);
}

int main(int argc, char *argv[])
{
    // THE_AGE 접근 테스트(간접 접근)
    log_info("My name: %s, age: %d", MY_NAME, get_age());

    // 나이 변경
    set_age(100);

    log_info("My age is now: %d", get_age());

    log_info("THE_SIZE is: %d", THE_SIZE);
}
```

```
// ex22.c 내부에서 바라보는 값 출력
print_size();

// 전역 변수 값 변경
THE_SIZE = 9;

log_info("THE SIZE is now: %d", THE_SIZE);

// 변경된 값 확인
print_size();

// 함수 내부 static 변수 동작 확인
log_info("Ratio at first: %f", update_ratio(2.0));
log_info("Ratio again: %f", update_ratio(10.0));
log_info("Ratio once more: %f", update_ratio(300.0));

// 스코프 테스트
int count = 4;

scope_demo(count);
scope_demo(count * 20);

// 함수 내부에서 변경해도 여기서의 count 는 영향받지 않음
log_info("count after acalling scope_demo: %d", count);

return 0;
}
```

Exercise 23. Duff's Device 만나보기

이 장은 뇌를 살짝 자극하는 퍼즐 같은 내용이다. 여기서 나는 C 언어에서 가장 유명한 해킹 기법 중 하나인 Duff's Device 를 소개한다. 이름은 발명자인 Tom Duff 에서 따왔다. 이 작은 코드 조각에는 지금까지 배운 거의 모든 개념이 한꺼번에 들어 있다.

이 코드가 어떻게 동작하는지 이해하는 것도 재미있는 퍼즐이 될 것이다.

주의

C 언어의 재미 중 하나는 이런 기발한 해킹을 만들어 낼 수 있다는 것이지만, 동시에 C 를 까다롭게 만드는 이유이다. 이런 트릭을 배우는 목적은 언어와 컴퓨터를 더 깊이 이해하기 위함이지, 실제로 이런 코드를 실무에서 쓰라는 뜻은 아니다.

항상 읽기 쉽고 유지보수하기 쉬운 코드를 작성하는 습관을 들이는 것이 중요하다.

[23-1] Duff's Device 소개

Tom Duff 가 발견한 이 기법은 C 컴파일러의 한계를 이용한 트릭이다. 사실은 제대로 동작하면 안되는 데, 이상하게도 돌아가는 구조이다.

아직 이 코드가 정확히 무엇을 하는지는 말하지 않는다. 먼저 코드를 실행해보고, 왜 이런 방식으로 작성되었는지 고민해본다.

[23-2] ex23. c 코드

```
#include <stdio.h>
#include <string.h>
#include "dbg.h"

// 일반적인 복사 함수
int normal_copy(char *from, char *to, int count) {
    int i = 0;
    for (i = 0; i < count; i++) {
        to[i] = from[i];
    }
    return i;
}

// Duff's Device 를 이용한 복사
int duffs_device(char *from, char *to, int count) {
    int n = (count + 7) / 8; // 8 개씩 묶어서 처리
    switch (count % 8) {     // 나머지 개수 처리
        case 0: do { *to++ = *from++;
        case 7:      *to++ = *from++;
```

```

        case 6:      *to++ = *from++;
        case 5:      *to++ = *from++;
        case 4:      *to++ = *from++;
        case 3:      *to++ = *from++;
        case 2:      *to++ = *from++;
        case 1:      *to++ = *from++;
                    } while (--n > 0);
    }
    return count;
}

// Zed 가 만든 Duff's Device 버전 (goto 사용)
int zeds_device(char *from, char *to, int count) {
    int n = (count + 7) / 8;
    switch (count % 8) {
        case 0:
            again: *to++ = *from++;
        case 7:  *to++ = *from++;
        case 6:  *to++ = *from++;
        case 5:  *to++ = *from++;
        case 4:  *to++ = *from++;
        case 3:  *to++ = *from++;
        case 2:  *to++ = *from++;
        case 1:  *to++ = *from++;
                if (--n > 0) goto again;
    }
    return count;
}

// 복사된 데이터가 예상 값과 일치하는지 검증
int valid_copy(char *data, int count, char expects) {
    for (int i = 0; i < count; i++) {
        if (data[i] != expects) {
            log_err("[%d] %c != %c", i, data[i], expects);
            return 0;
        }
    }
    return 1;
}

```

```

int main(int argc, char *argv[]) {
    char from[1000];
    char to[1000];
    int rc = 0;

    // 초기값 설정
    memset(from, 'x', 1000); // from 배열을 'x'로 채움
    memset(to, 'y', 1000);   // to 배열을 'y'로 채움
    check(valid_copy(to, 1000, 'y'), "초기화가 올바르게 안됨.");

    // 일반 복사
    rc = normal_copy(from, to, 1000);
    check(rc == 1000, "Normal copy 실패: %d", rc);
    check(valid_copy(to, 1000, 'x'), "Normal copy 실패");

    // 초기화
    memset(to, 'y', 1000);

    // Duff's Device 복사
    rc = duffs_device(from, to, 1000);
    check(rc == 1000, "Duff's Device 실패: %d", rc);
    check(valid_copy(to, 1000, 'x'), "Duff's Device 복사 실패");

    // 초기화
    memset(to, 'y', 1000);

    // Zed 버전 복사
    rc = zeds_device(from, to, 1000);
    check(rc == 1000, "Zed's Device 실패: %d", rc);
    check(valid_copy(to, 1000, 'x'), "Zed's Device 복사 실패");

    return 0;

error:
    return 1;
}

```

[23-3] 코드 설명

1. normal_copy: 단순 for 루프로 from → to 복사
2. duffs_device: switch + do-while 을 이용한 Duff's Device. 8 개 단위로 루프 언롤링
3. zeds_device: goto 를 사용한 Duff's Device. 구조를 더 직관적으로 이해 가능
4. valid_copy: 배열이 기대값과 일치하는지 확인

[23-4] Duff's Device 원리

- 루프 언롤링: 한 번의 루프에서 여러 반복을 처리해 속도를 높임
- switch fallthrough: break 없이 다음 case 로 넘어가는 특성 이용
- count % 8: 루프 시작 전에 남은 나머지 처리
- goto 버전: do-while 대신 goto 를 사용해서 루프 구조를 명시적으로 표현

[23-4] 연습 방법

- 배열 크기를 바꾸어 동작 확인(1000 → 10, 15, 20)
- count % 8 값과 n 을 추적하면서 각 반복이 어떻게 진행되는지 확인
- zeds_device 와 비교하면서 Duff's Device 구조 이해
- valid_copy 로 각 방법이 정상적으로 복사되는지 확인

Duff's Device 코드 정리

[1] 프로그램의 목적

이 프로그램은 배열 복사를 세 가지 방식으로 구현하고 비교하는 예제이다.

1. 일반적인 for 문을 이용한 복사(normal_copy)
2. Duff's Device 를 이용한 복사(duffs_device)
3. goto 를 사용한 Duff 변형(zeds_device)

이 코드의 핵심 목적은 언롤링 기법을 이해하는 것이다.

루프 언롤링(Loop Unrolling)

반복문(loop)의 횟수를 줄이기 위해 내부 코드를 직접 나열하여, 제어 코드 오버헤드를 제거하고 실행 속도를 최적화하는 기법이다.

[2] 전체 실행 흐름

main 함수의 전체 동작 과정은 다음과 같다.

1. from 배열을 'x'로 초기화
2. to 배열을 'y'로 초기화
3. normal_copy 로 복사 후 검증
4. duffs_device 로 복사 후 검증
5. zeds_device 로 복사 후 검증

각 복사 후에는 valid_copy 함수를 이용해 데이터가 정확히 복사되었는지 확인한다.

[3] normal_copy 함수

```
int normal_copy(char *from, char *to, int count) {  
    int i = 0;  
    for (i = 0; i < count; i++) {  
        to[i] = from[i];  
    }  
    return i;  
}
```

[3-1] 매개변수 의미

```
int normal_copy(char *from, char *to, int count)
```

- from : 복사할 원본 배열
- to: 복사 대상 배열
- count: 복사할 데이터 개수

[3-2] 동작 코드

```
for (i = 0; i < count; i++) {  
    to[i] = from[i];  
}
```

동작 원리

- 0 부터 count -1 까지 반복
- 한 번에 1 개씩 복사
- 가장 기본적이고 이해하기 쉬운 방식

[3-3] 특징

- 가독성이 매우 좋음
- 반복 횟수가 많음(count 번 반복)
- 매 반복마다 조건 검사 발생

[4] Duff's Device 함수

```
int duffs_device(char *from, char *to, int count) {  
    int n = (count + 7) / 8; // 8 개씩 묶어서 처리  
    switch (count % 8) {      // 나머지 개수 처리  
        case 0: do { *to++ = *from++;  
        case 7:      *to++ = *from++;  
        case 6:      *to++ = *from++;  
        case 5:      *to++ = *from++;  
        case 4:      *to++ = *from++;  
        case 3:      *to++ = *from++;  
        case 2:      *to++ = *from++;  
        case 1:      *to++ = *from++;  
        } while (--n > 0);  
    }  
    return count;  
}
```

[4-1] 매개변수

```
int duffs_device(char *from, char *to, int count)
```

Duff's Device 는 루프 언롤링을 이용해 반복 횟수를 줄이는 기법이다.

[4-2] 반복 횟수 계산

```
int n = (count + 7) / 8;
```

7을 더하는 이유는 8 개씩 묶어서 처리하기 때문에 반복 횟수를 올림 계산해야 한다.
정수 나눗셈은 소수점을 버리기 때문에 단순히

$\text{count} / 8$

을 하면 나머지가 무시된다.

따라서 올림 계산을 위해 다음 공식을 사용한다.

$(\text{count} + k - 1) / k$

여기서 $k = 8$ 이므로:

$(\text{count} + 7) / 8$

예시:

- 1000 개: 125 번 반복
- 1003 개 : 126 번 반복

[4-3] switch 의 역할

$\text{switch}(\text{count} \% 8)$

왜 나머지를 구하냐면 8 개씩 묶으면 남는 개수가 생길 수 있다.

예를 들어

$1003 \% 8 = 3$

즉, 3 개가 남는다.

switch 는 루프의 시작 위치를 조정하는 역할을 한다.

[4-4] 핵심 구조

```
case 0:
    do{
        case 7: *to++ = *from++;
        case 6: *to++ = *from++;
        case 5: *to++ = *from++;
        case 4: *to++ = *from++;
        case 3: *to++ = *from++;
        case 2: *to++ = *from++;
        case 1: *to++ = *from++;
    }while (--n > 0);
```

fall-through

break 가 없기 때문에 case 아래로 계속 실행된다.

$*to++ = *from++;$

$*(to++) = *(from++);$

동작 순서

1. `*(from++)`
 - `from` 이 가리키는 메모리 주소 안의 값을 가져옴
 - 그 다음 `from` 을 다음 주소로 증가
2. `*(to++)`
 - `to` 가 가리키는 메모리 주소 안에 값 저장
 - 그 다음 `to` 를 다음 주소로 증가

[4-5] 실제 동작 예시(count = 1003)

1. `n = 126`
2. `count % 8 = 3`
3. case 3 부터 시작

첫 번째 반복

case 3 → case2 → case1 : 총 3 개 복사

이후 반복

do-while 에 의해 다시 위로 돌아감

이번에는 case 0 부터 시작 : 매 반복마다 8 개씩 반복

총 계산

$$3 + (125 \times 8) = 1003$$

[5] `zeds_device` 함수

```
int zeds_device(char *from, char *to, int count) {
    int n = (count + 7) / 8;
    switch (count % 8) {
        case 0:
            again: *to++ = *from++;
        case 7:  *to++ = *from++;
        case 6:  *to++ = *from++;
        case 5:  *to++ = *from++;
        case 4:  *to++ = *from++;
        case 3:  *to++ = *from++;
        case 2:  *to++ = *from++;
        case 1:  *to++ = *from++;
                if (--n > 0) goto again;
    }
    return count;
}
```

```
}
```

[5-1] zeds_device 설명

```
again:
```

```
    *to++ = *from++;
```

```
...
```

```
if (--n > 0) goto again;
```

- do-while 대신 goto 사용
- 동작 논리는 Duff's Device 와 동일
- 가독성은 더 낮음
- 학습 목적의 코드

[6] valid_copy 함수

```
int valid_copy(char *data, int count, char expects) {  
    for (int i = 0; i < count; i++) {  
        if (data[i] != expects) {  
            log_err("[%d] %c != %c", i, data[i], expects);  
            return 0;  
        }  
    }  
    return 1;  
}
```

역할

- 배열을 처음부터 끝까지 검사
- 모든 값이 expects 와 같은지 검사

반환값

- 모두 같으면 1
- 하나라도 다르면 0

[7] 최종 스어리

Duff's Device 는 switch 와 do-while 을 결합하여 루프의 시작 위치를 조정하고 첫 반복에서 나머지를 처리한 뒤 이후에는 8 개씩 묶어서 처리하는 기법이다.

현대 컴파일러에서는 자동 최적화를 해주기 때문에

실무에서 직접 사용할 일은 거의 없지만,

C 언어의 동작 원리와 저수준 최적화를 이해하는 데 매우 중요한 예제이다.

Exercise 24. 입력, 출력, 파일

지금까지 우리는 `printf` 를 사용해서 화면에 출력했다. 그건 좋지만, 이제는 더 많은 기능이 필요하다. 이번 장에서는 `fscanf` 와 `fgets` 함수를 사용해서 사람의 정보를 구조체에 저장하는 프로그램을 만든다.

이 간단한 입력 읽기 소개가 끝나면, C 에서 제공하는 I/O 함수들의 전체 목록도 살펴본다. 이미 몇 개는 사용해봤겠지만, 이번에는 일종의 암기 훈련이라고 생각하면 된다.

[24-1] `ex24.c` 코드